

Register Number: 192425231

Student Name: VANJINATHAN S

Q38. Edge Coloring Problem (NP-Complete Problem)

1. Introduction

Edge coloring is a classic problem in graph theory in which we have to assign colors to the edges of a graph such that no two edges that meet at the same vertex (adjacent edges) get the same color. The main aim is to use the least possible number of colors while satisfying this rule. This minimum number of colors required is known as the chromatic index of the graph, denoted by $\chi'(G)$. The Edge Coloring Problem is an NP-Complete problem because, for a general graph, deciding whether the graph can be properly edge colored using a given number of colors k is computationally hard, and no polynomial time algorithm is known to solve it for all cases.

2. Problem Definition

Given an undirected graph $G = (V, E)$, where V is the set of vertices and E is the set of edges, the edge coloring problem is to find a coloring function $C: E \rightarrow \{1, 2, 3, \dots, k\}$ such that for any two edges e_1 and e_2 that share a common vertex, $C(e_1) \neq C(e_2)$. The problem is to determine the smallest value of k for which such a coloring exists.

3. Constraints Related to Adjacency and Color Limits

3.1 Adjacency Constraint

Two edges are said to be adjacent if they share a common endpoint (vertex). The main constraint of this problem is that any two adjacent edges must never be assigned the same color. This ensures that at every vertex, all the edges meeting there have different colors.

3.2 Degree Constraint

At any vertex v , if the degree of that vertex (number of edges connected to it) is $d(v)$, then at least $d(v)$ different colors are required just to color the edges meeting at that vertex. So the maximum degree of the graph, denoted as $\Delta(G)$, gives a lower bound on the number of colors needed.

3.3 Color Limit Constraint (Vizing's Theorem)

According to Vizing's Theorem, the chromatic index of any simple graph lies between $\Delta(G)$ and $\Delta(G) + 1$. That means:

$$\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1$$

This theorem gives an upper limit for the number of colors that will ever be needed, and it forms the basis for most edge coloring algorithms.

4. How Constraints Influence Feasible Assignments

The adjacency and degree constraints directly affect which color assignments are feasible for a given graph:

- If a vertex has a high degree, more colors are forced to be used only for that vertex, which increases the overall number of colors needed for the whole graph.
- Graphs where every vertex has the same degree (regular graphs) tend to need exactly $\Delta(G)$ colors (Class 1), while some graphs need $\Delta(G) + 1$ colors (Class 2).
- Dense graphs with many shared vertices between edges create more adjacency restrictions, which reduces the number of feasible colorings available and increases the search space for a valid solution.
- Because of these growing restrictions as the graph size increases, checking whether a graph can be colored with exactly k colors becomes computationally expensive, which is the reason this problem is classified as NP-Complete.

5. Development of the Algorithm

A simple approach to solve the edge coloring problem is by using a Greedy / Backtracking method. The steps are given below:

1. List all the edges of the graph $E = \{e_1, e_2, e_3, \dots, e_n\}$.
2. Find the maximum degree $\Delta(G)$ of the graph, and set the possible number of colors as $k = \Delta(G)$ or $\Delta(G) + 1$ (based on Vizing's theorem).
3. Take the first uncolored edge and assign it the smallest available color that is not already used by any of its adjacent edges.
4. Move to the next edge and repeat step 3.
5. If at any point an edge cannot be assigned any of the k colors without conflict, backtrack to the previous edge and try a different color.
6. Continue this process until all the edges are colored, or report that a valid coloring is not possible with the chosen k .

Pseudocode

```
function EdgeColoring(Graph G, int k):  
    for each edge e in G.edges:  
        for color c = 1 to k:  
            if no adjacent edge of e uses color c:
```

```
        assign color c to e
        break
    if e has no valid color:
        backtrack to previous edge
return coloring assignment
```

6. Example

Consider a simple graph with 4 vertices A, B, C, D forming a cycle: A-B, B-C, C-D, D-A. Here every vertex has degree 2, so $\Delta(G) = 2$. Using the algorithm, we can color edge A-B with Color 1, edge B-C with Color 2, edge C-D with Color 1, and edge D-A with Color 2. No two adjacent edges share the same color, so this graph is properly edge colored using only 2 colors, which matches the lower bound $\Delta(G)$.

7. Time Complexity

The greedy edge coloring approach takes $O(V + E)$ time to produce a coloring using at most $\Delta(G) + 1$ colors, but this does not always give the minimum possible colors. To check whether the exact chromatic index ($\Delta(G)$ or $\Delta(G) + 1$) can be achieved requires solving an NP-Complete decision problem, which does not have a known polynomial time solution for general graphs. Hence exact algorithms use backtracking or exponential time methods for optimal results.

8. Conclusion

The Edge Coloring Problem shows how a simple looking rule, that adjacent edges cannot share a color, leads to a computationally difficult problem when applied to large or dense graphs. The adjacency constraint and the degree of vertices together decide how many colors are feasible, and Vizing's theorem gives a tight bound between $\Delta(G)$ and $\Delta(G) + 1$. Because verifying the minimum number of colors is hard for general graphs, the problem is classified as NP-Complete, and practical solutions rely on greedy and backtracking based algorithms.